

Security and Quality Review

Executive assessment

The repository is a credible demonstration project, not a production-ready booking or payment system. Authentication and role middleware exist, but a public caller can register as **ADMIN**, making the authorization model ineffective. Booking capacity also lacks atomicity. These are release blockers.

Findings

ID	Severity	Finding	Evidence/impact	Recommended correction
SEC-01	Critical	Public privilege escalation	Registration accepts and persists caller-provided ADMIN/ORGANIZER ; UI exposes both	Ignore public role ; provision privileged accounts through protected admin workflow
SEC-02	Critical	Hard-coded JWT fallback secret	Same source-known fallback is used for signing and verification	Fail startup if JWT_SECRET is missing; store/rotate secret securely
DATA-01	High	Non-atomic seat allocation	Slot is read, checked, decremented, and saved separately; concurrent holds can conflict/oversell	Conditional \$inc with availability predicate plus transaction/idempotency
DATA-02	High	Seat leak on failure/cancel	Seats are decremented before booking creation; errors do not roll back; pending cancellation does not restore	Transaction and state-aware, idempotent release logic
AUTH-01	High	Organizer access is global	No assignment field; organizer can manage any slot and query all records	Store assigned templates and enforce them in every query/mutation
AUTH-02	High	Booking role enforcement relies partly on UI	Backend permits any authenticated role to create bookings/reviews	Encode allowed roles and ownership rules in server middleware/services
PAY-01	High	Payment is only simulated	Confirmation endpoint accepts no gateway proof; revenue analytics treats records as revenue	Label demo clearly or integrate provider with server verification/webhooks
SEC-03	High	Missing request validation/sanitization	Direct API calls bypass UI limits; regex input is accepted directly	Use DTO/schema validation, safe regex handling, normalization, and limits

ID	Severity	Finding	Evidence/impact	Recommended correction
SEC-04	Medium–High	No rate limiting or abuse controls	Login, registration, reviews, booking holds, and donations are unthrottled	Add per-IP/account limits, lockout/backoff, and anti-automation controls
SEC-05	Medium	JWT stored in <code>localStorage</code>	Successful XSS can extract a 30-day token	Strong CSP/XSS prevention; consider secure HttpOnly SameSite cookies and shorter sessions
SEC-06	Medium	Open CORS	<code>cors()</code> accepts broad defaults	Allowlist trusted origins/methods/headers and credentials policy
DATA-03	Medium–High	Delete integrity mismatch	Temple delete removes only temple, although UI claims related data is removed	Implement soft delete/cascade/retention policy and truthful confirmation text
DATA-04	Medium	Weak identifier strategy	Ticket collision raises error without retry; transaction ID lacks unique index	Use robust unique IDs and retry on duplicate-key errors
OPS-01	Medium	Expiration depends on traffic	Holds expire only when selected APIs trigger a sweep	Scheduled worker/TTL-assisted design with idempotent release
OPS-02	Medium	Health check is shallow	<code>/health</code> reports healthy even when MongoDB is unavailable	Separate liveness and readiness endpoints
QUAL-01	High	No automated tests	Critical booking/access logic has no regression coverage	Add unit/integration/E2E and concurrency gates
QUAL-02	Medium	Frontend route guards absent	Wrong-role users can render dashboard shells and confusing failures	Add loading-aware role guards; keep API enforcement authoritative
QUAL-03	Medium	Error mapping is inconsistent	Validation/cast/duplicate errors commonly return 500 and raw messages	Central error taxonomy; avoid exposing internal database details
REPO-01	Low–Medium	Dependencies committed in server	<code>server/node_modules</code> increases repository size/noise and supply-chain ambiguity	Remove from version control; install from lockfile in CI/deploy

Additional observations

- The client-side math captcha is not a security control because clients can bypass it and call the API directly.
- No CSP, Helmet, audit trail, CSRF design, account verification, password reset, MFA, or token revocation is present.
- No application code sends the entered card number/CVV to the server, which avoids storing them in the current implementation; the UI must still state clearly that checkout is simulated.
- Password length is checked only in the registration UI. The schema/API does not impose the same minimum.
- Admin user update can set roles and passwords, which is powerful and should be audited and constrained.
- The “last admin” guard protects the single remaining admin from deleting that account, but privileged user changes still lack audit logging and stronger approval controls.

Recommended remediation order

1. Close public privilege escalation and require secrets.
2. Make booking allocation, failure rollback, expiration, and cancellation consistent.
3. Introduce organizer assignments and server-side validation/authorization.
4. Define payment as explicitly simulated or integrate a compliant gateway.
5. Add rate limiting, secure headers/CORS/session design, error normalization, and audit logging.
6. Add automated tests and CI gates, then address lint/accessibility/operational gaps.

Confidence

Confidence is high for source-visible findings at commit [d1c1a5f](#). Runtime/infrastructure findings are limited because the repository supplies no deployed environment, secrets policy, MongoDB service, or CI configuration.